Modern Storages and Data Warehousing
Week 6 - Queues&CDC

Попов Илья, i.popov@hse.ru

Recap прошлых занятий

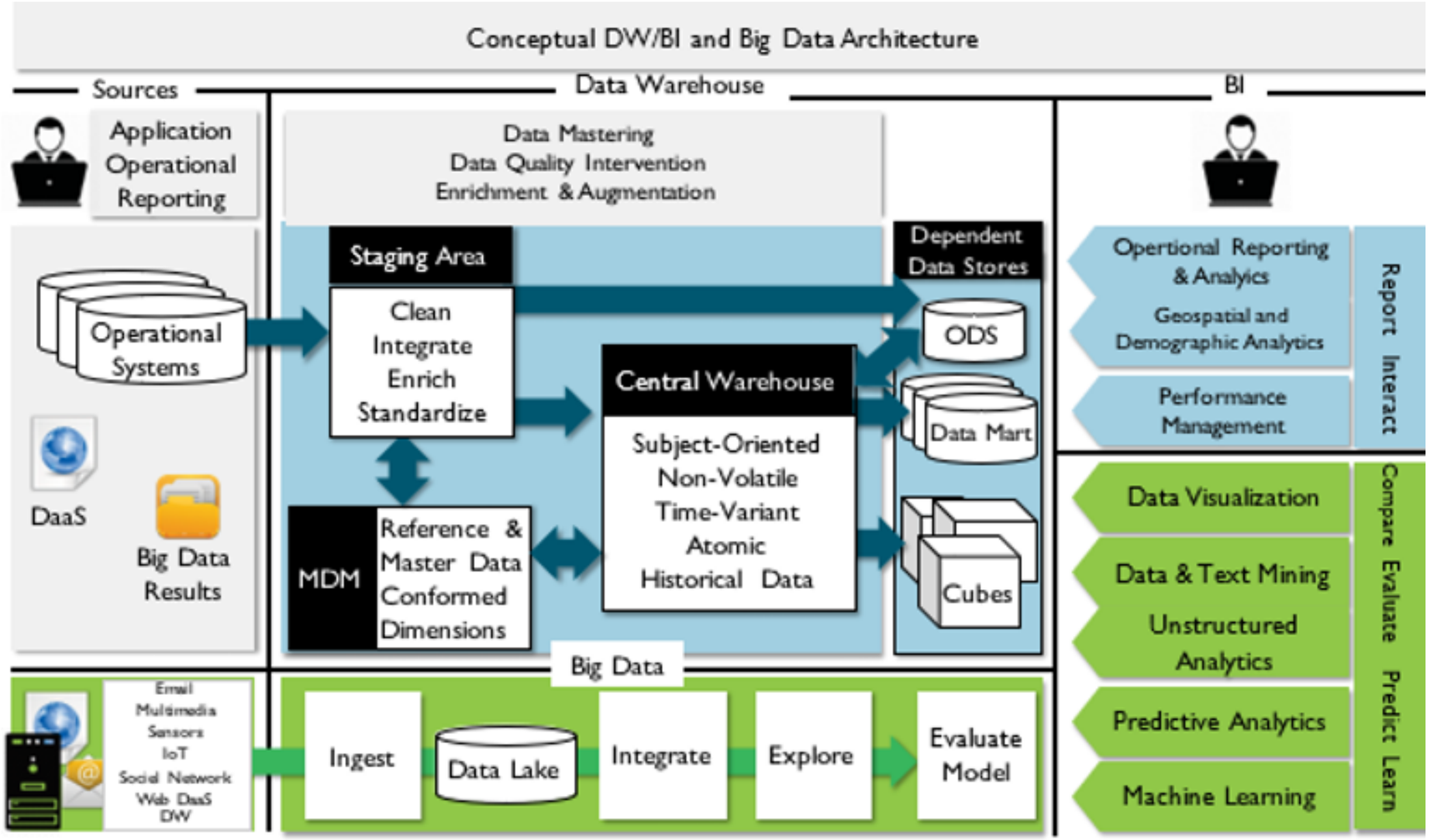


Figure 5: Date Warehouse Concept

1 - Очереди

Концепция Message broker

Message broker - архитектурный паттерн в распределённых системах. Это промежуточное ПО, которое развязывает producer и consumer во времени и в пространстве - оно принимает, хранит и маршрутизирует сообщения.

Брокеры позволяют создать буфер, который может коммуницировать с различными системами по унифицированным протоколам, создавать канал коммуникации между приложениями или системами.

Концепция Message broker

Две модели обмена сообщениями в message broker’ax:

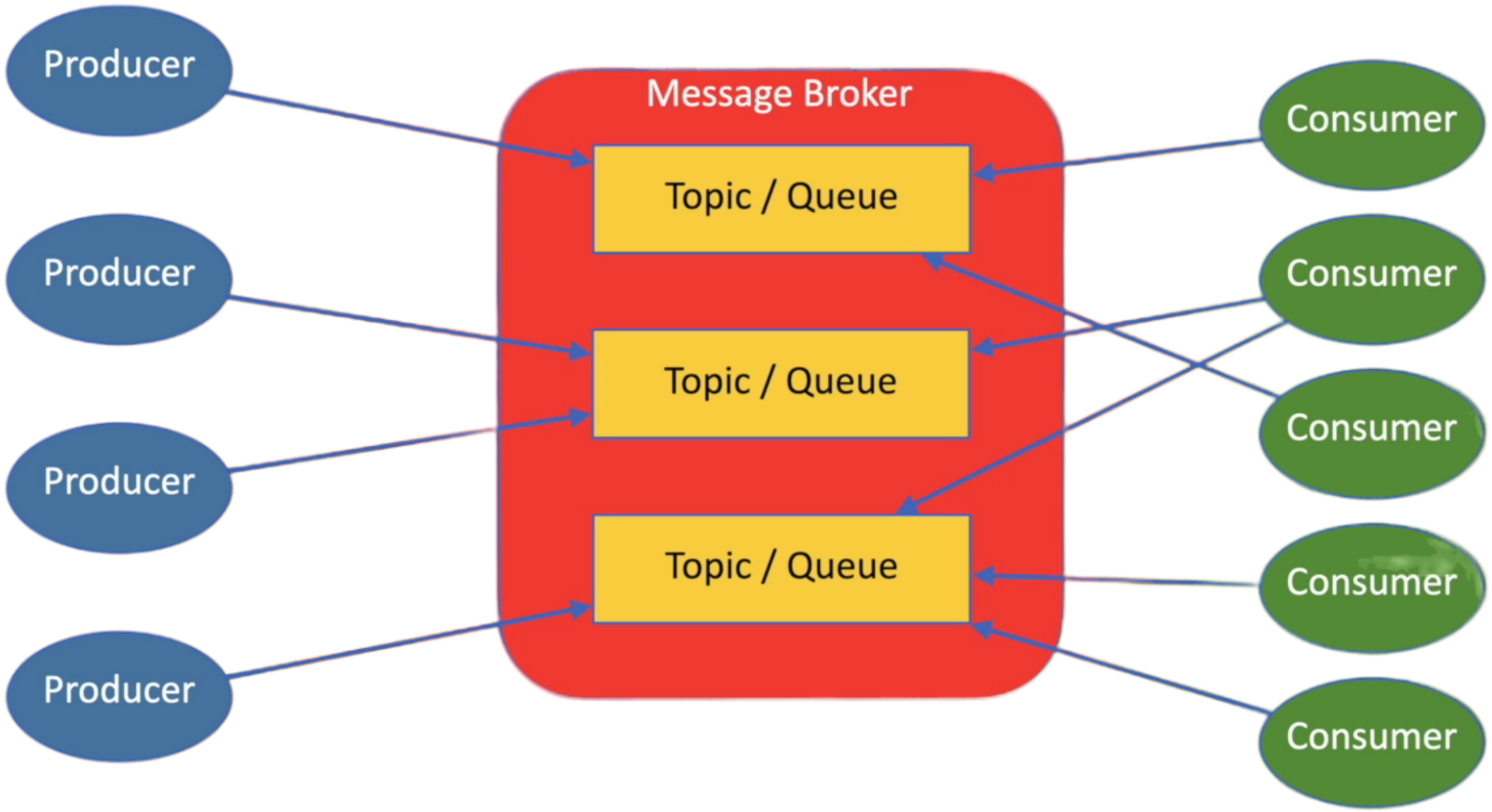
〉 **point-to-point**: брокеры, которые работают по принципу доставки конкретного сообщения. Применяется message-ориентированный подход - каждое сообщение получает ровно один получатель из группы. Выполняется в виде очереди FIFO, в которую одна система пишет сообщения, а другая система вычитывает эти сообщения;

〉 **publish/subscribe**: источники (producer-ы) публикуют свои изменения, а потребители (consumer-ы) получают эти сообщения по подписке. Нет гарантии строгой последовательности, но являются более масштабируемыми.

Назначение Message broker

- **Интеграция систем с разными протоколами** – можем использовать различные языки, т.к. для большинства МВ существуют клиентские библиотеки, которые позволяют общаться с МВ
- **Роутинг сообщений** – можем настраивать правила отправки сообщений
- **Надёжное хранение** – при правильной настройке отправленное сообщение не будет потеряно
- **Гарантированная доставка** – гарантировано, что сообщение будет доставлено, но не все МВ гарантируют, что единожды отправленное сообщение не будет получено приёмником несколько раз
- **Масштабирование (как источников, так и потребителей)** – должны позволять подключать большее количество источников и потребителей
- **Преобразование сообщений** – сообщение может быть преобразовано внутри МВ (например, можно шифровать, маскировать сообщения)
- **Интеграция с внешними системами** – МВ может выступать не просто как канал передачи данных, но и взаимодействовать с внешними системами, и по результату каким-то образом проверить, обогатить, правильно маршрутизировать сообщение

Общая схема Message broker

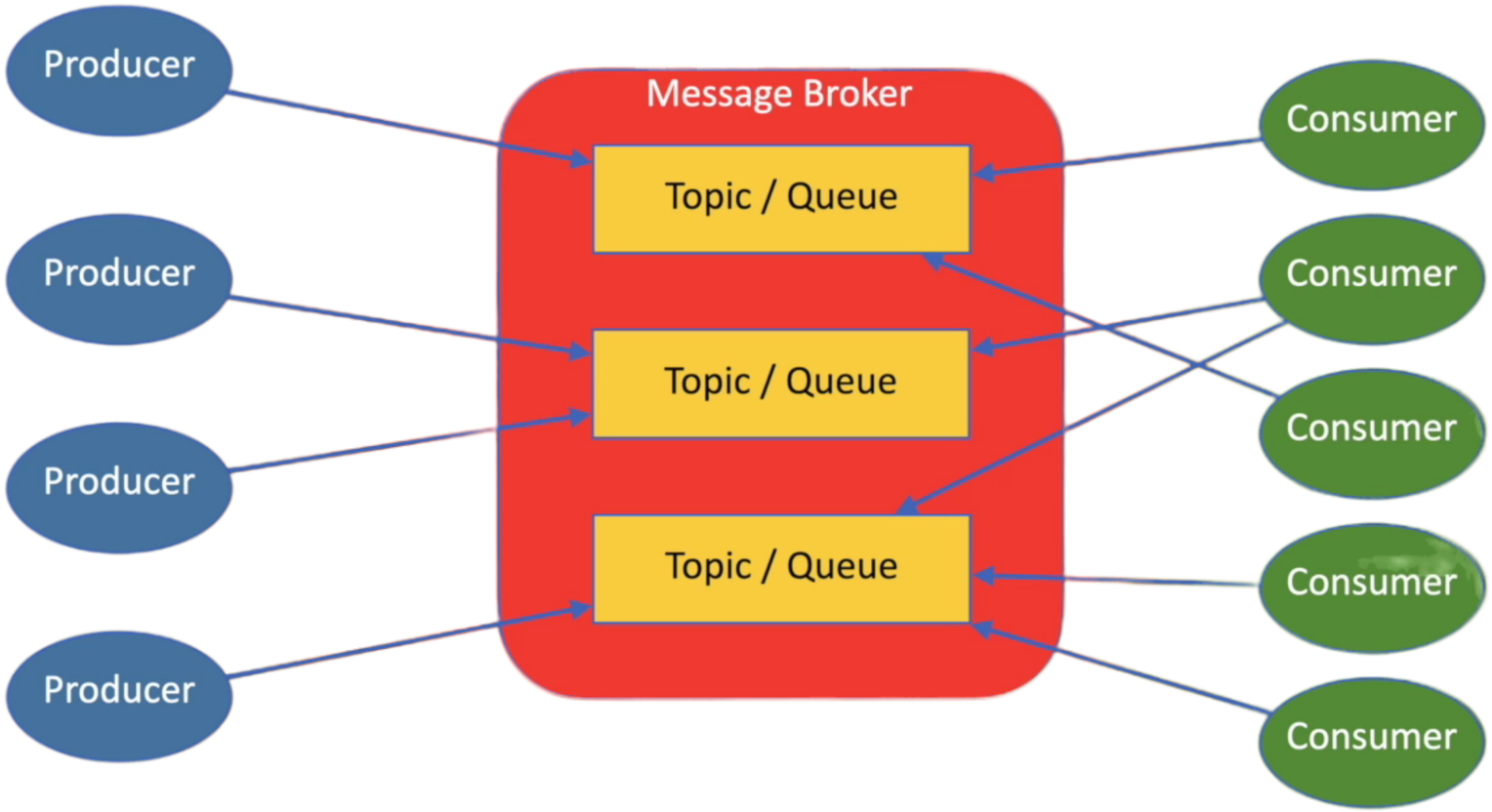


〉 **Topic / Queue** – логический канал передачи информации

〉 **Producer** – системы-источники информации подключаются к MB и пишут свои сообщения в Topic (Queue)

〉 **Consumer** – потребители сообщений. Способ получения (push или pull) зависит от брокера. Например, RabbitMQ доставляет push'ем, а Kafka – pull’ом

Общая схема Message broker



- Несколько Producer-ов могут писать в один Topic / Queue
- Несколько Consumer-ов могут получать данные из одного Topic / Queue
- Consumer может подписаться на несколько Topic / Queue

Apache Kafka



- Изначально - внутренний проект в LinkedIn
- Вышла в opensource в 2011 году
- Названа в честь Франца Кафки ¯_(ツ)_/¯

Apache Kafka



- Изначально - внутренний проект в LinkedIn
- Вышла в opensource в 2011 году
- Названа в честь Франца Кафки ¯_(ツ)_/¯

- Работает по модели publish/subscribe
- Внутри представляет собой распределённый append-only лог (event streaming platform). Pub/sub и очередь реализуются через consumer group’ы
- Имеет распределенную и гибко масштабируемую архитектуру

Apache Kafka

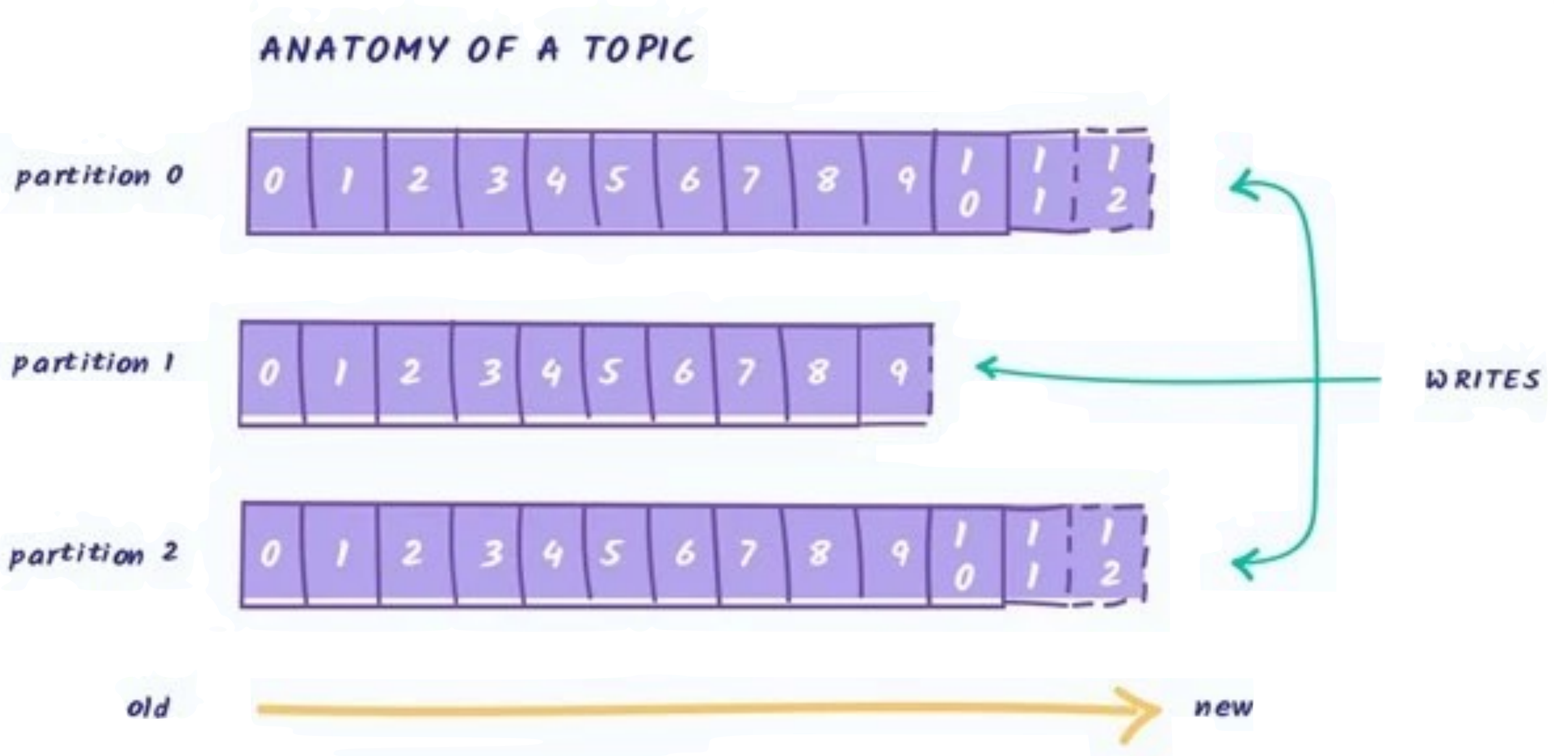


〉 Изначально - внутренний проект в LinkedIn
〉 Вышла в opensource в 2011 году
〉 Названа в честь Франца Кафки ¯_(ツ)_/¯

〉 Работает по модели publish/subscribe
〉 Внутри представляет собой распределённый append-only лог (event streaming platform). Pub/sub и очередь реализуются через consumer group’ы
〉 Имеет распределенную и гибко масштабируемую архитектуру

〉 Запись = key (опционально) + value (bytes) + headers + timestamp
〉 Сообщения группируются в topic
〉 Внутри топиков: key-value структура
〉 Настраиваются Log Retention и Cleanup Policy

Устройство Kafka

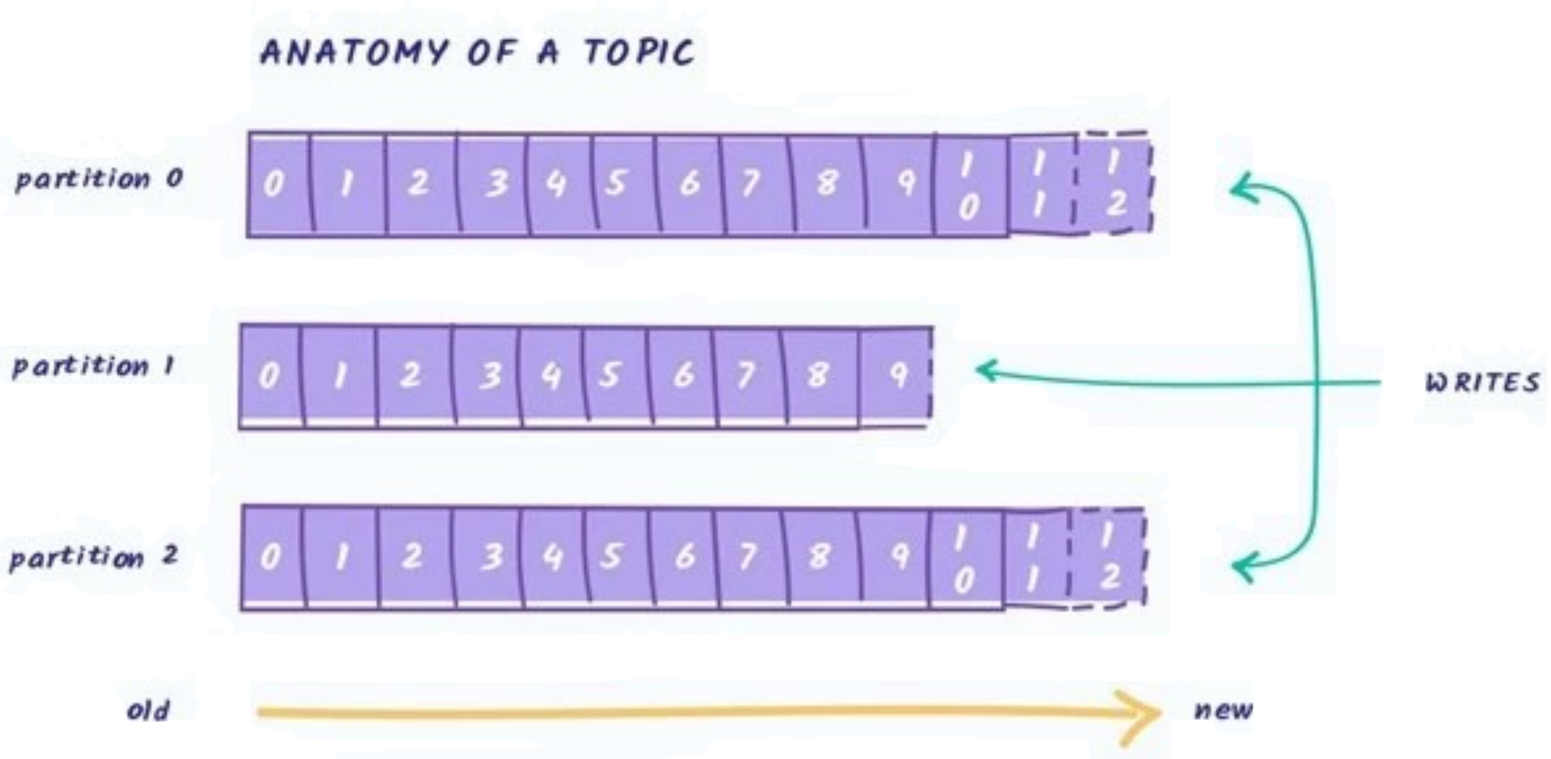


Topic:

〉 логическая append-only очередь сообщений (message): можем только добавлять сообщения, а удалить ошибочное сообщение не можем

〉 состоит из 1+ партиций (partition): для максимального распараллеливания обработки сообщений как с точки зрения записи, так и чтения

Устройство Kafka



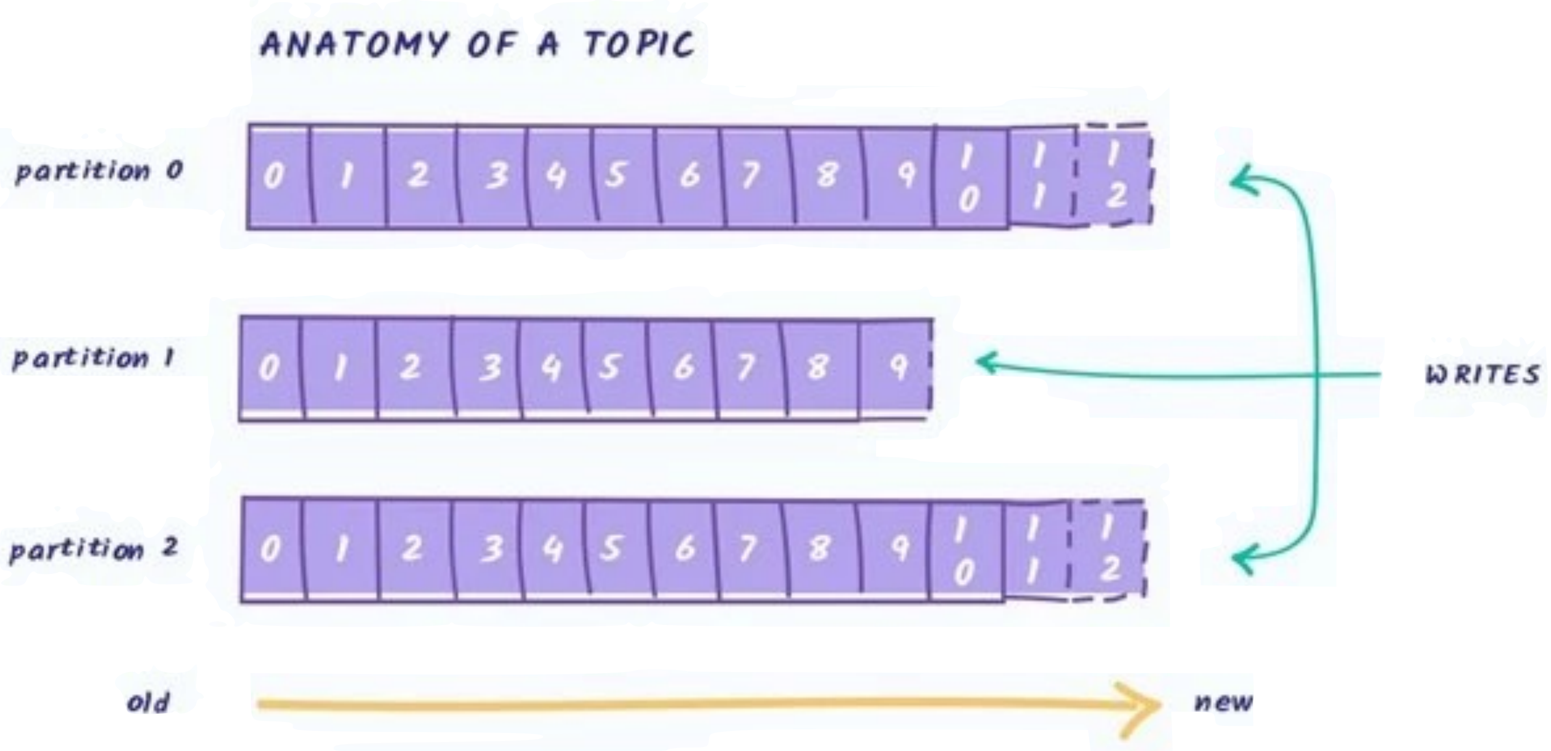
Partition:

〉 физическая единица хранения данных topic: состоит из файлов, которые хранятся на сервере

〉 в partition можно писать, но нельзя удалять

〉 в конкретный момент времени жёстко привязана к конкретному broker-у. Есть репликация, но только одна partition будет active, остальные follower

Устройство Kafka

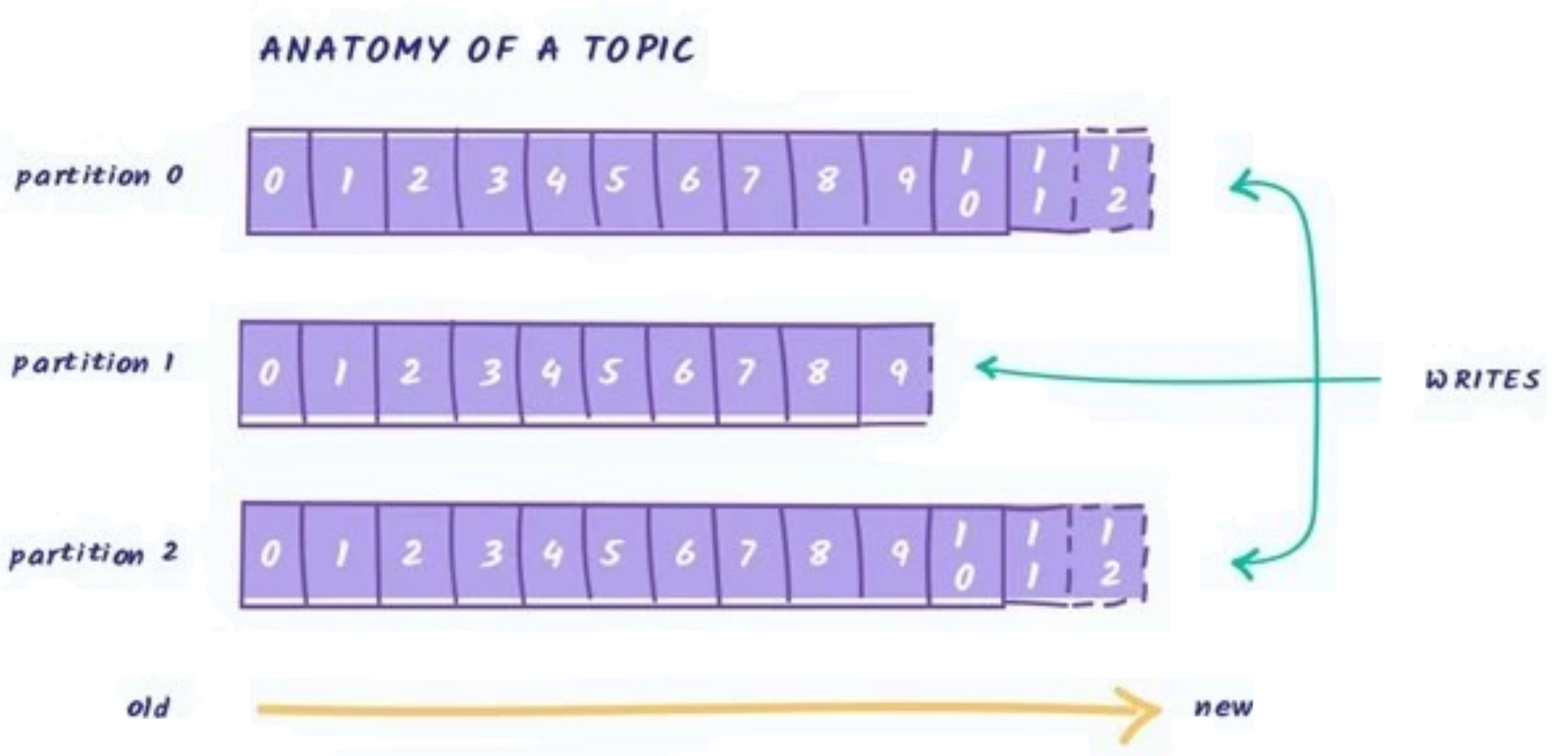


Запись и чтение из partition:

〉 Если используется ключ для сообщения, то для определения партиции используется hash(key) % кол-во партиций
Consumer, подключаясь к topic, понимает топологию
внутри topic и подключается к одной или нескольким partition

〉 Если не используется ключ для сообщения, то распределение между partition происходит по принципу round-robin – каждое следующее сообщение пишется в следующую partition и далее по кругу.

Устройство Kafka



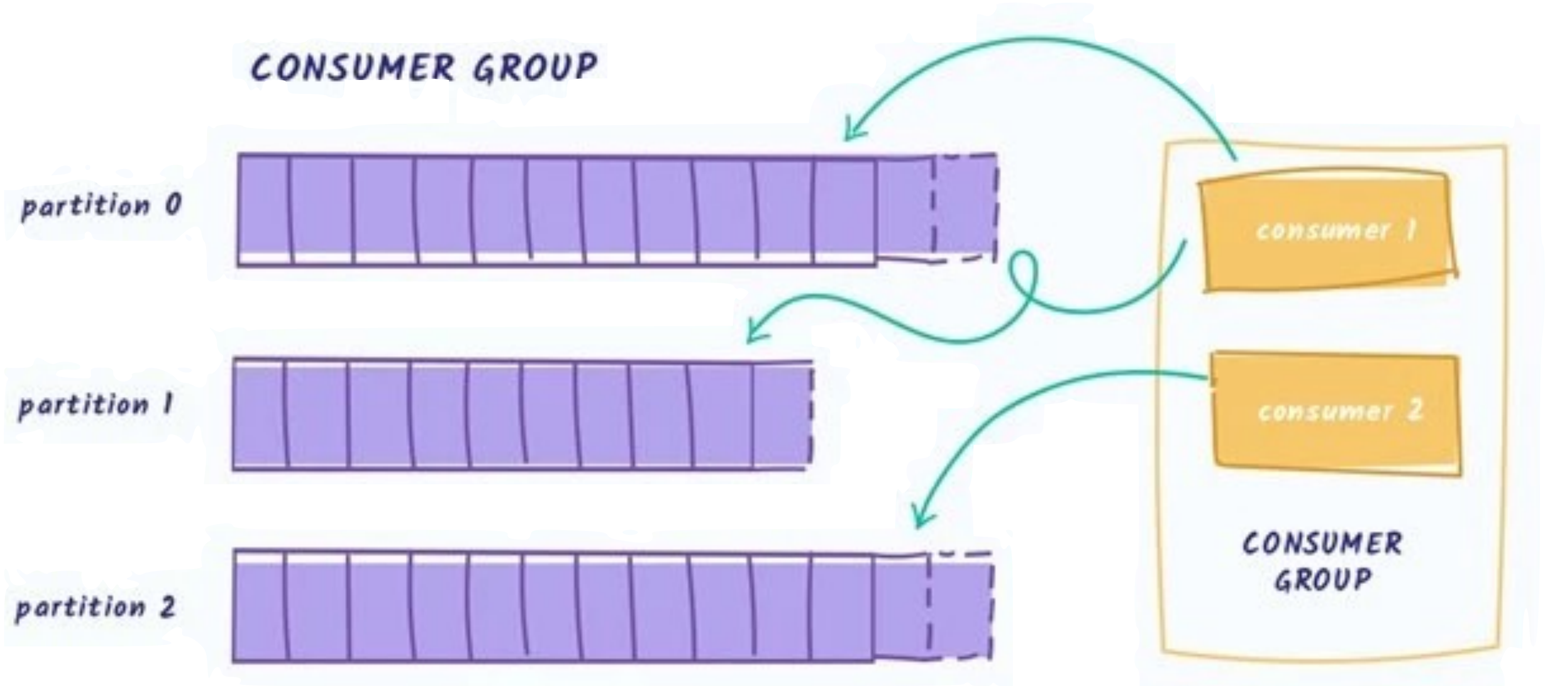
〉 Навигация по topic производится на основе offset - инкрементальный идентификатор события внутри каждой партиции

〉 Все сообщения нумеруются, и мы получаем сообщения в том порядке, в котором они были пронумерованы

〉 Offset не является глобальной величиной для topic, а работает на уровне партиции

〉 Если topic состоит из более чем одной партиции, то нет гарантии строгой последовательности получения сообщений

Устройство Kafka

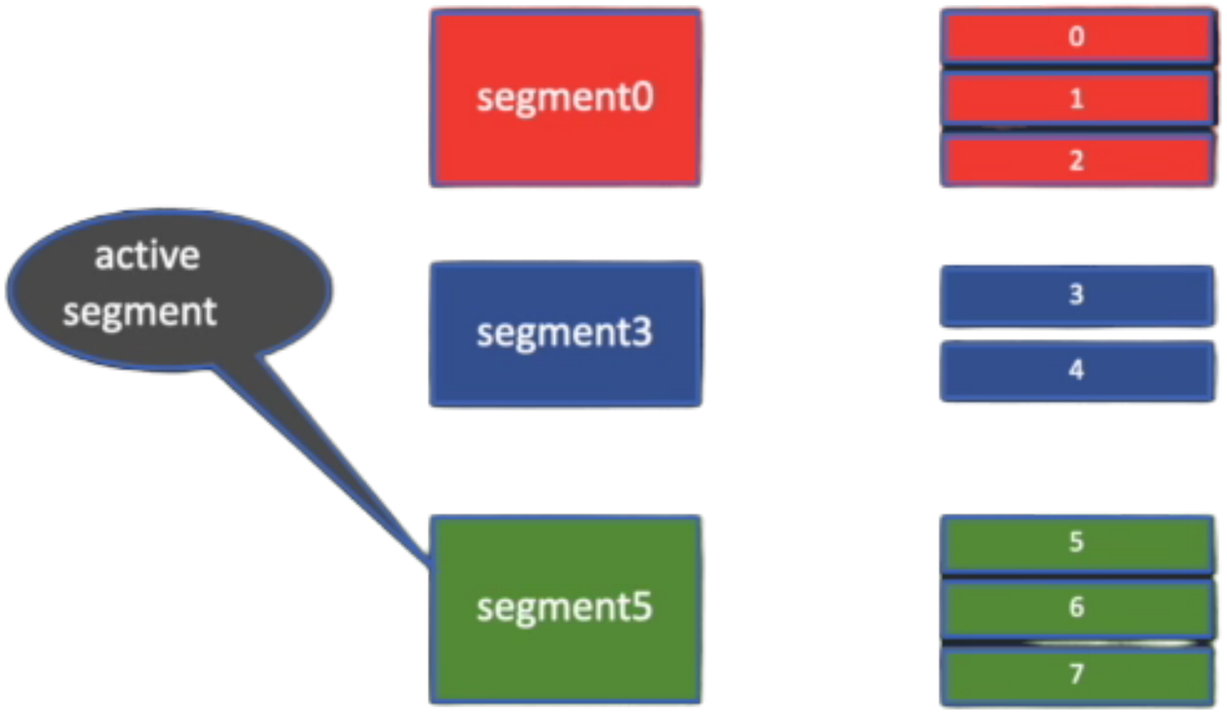


〉 Consumer’ы читают данные из партиций напрямую

〉 При чтении из партиции consumer делает **коммит оффсета**. Это необходимо для того, чтобы, если, например, текущий читатель упадёт, то следующий (новый читатель) начнёт с последнего коммита

〉 Consumer’ы объединяются в **consumer group**. При добавлении нового читателя или падении текущего, группа перебалансируются. Объединение в группы гарантирует, что сообщение внутри группы будет прочитано **at least once**

Устройство Kafka



Segment:

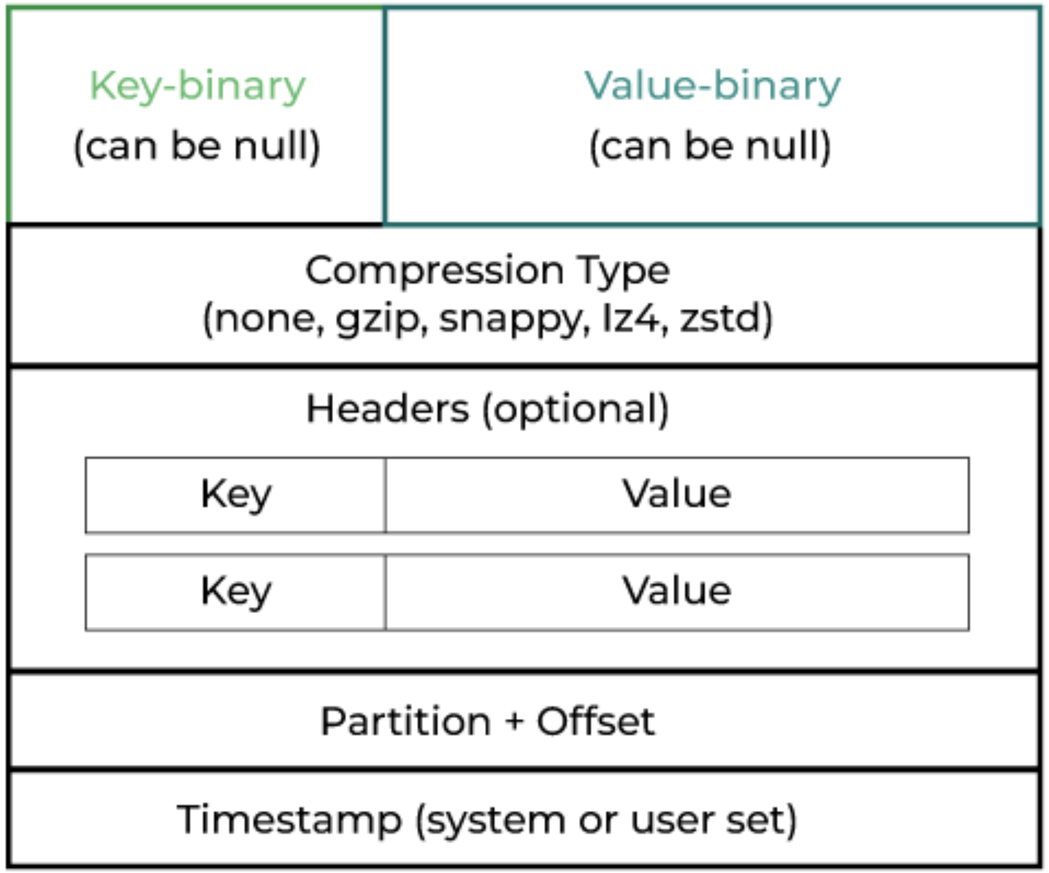
〉 Набор записей внутри partition

〉 Физически являются файлами в файловой системе

〉 Всегда есть один **active segment**, в который идёт запись новых message

〉 При превышении размера сегмента (или времени жизни) создаётся новый, который становится активным

Сообщение Kafka



Message:

〉 Record:

key (bytes, nullable)

value (bytes, nullable)

headers (список key: String → value: bytes)

timestamp (CreateTime или LogAppendTime)

〉 На уровне батча:

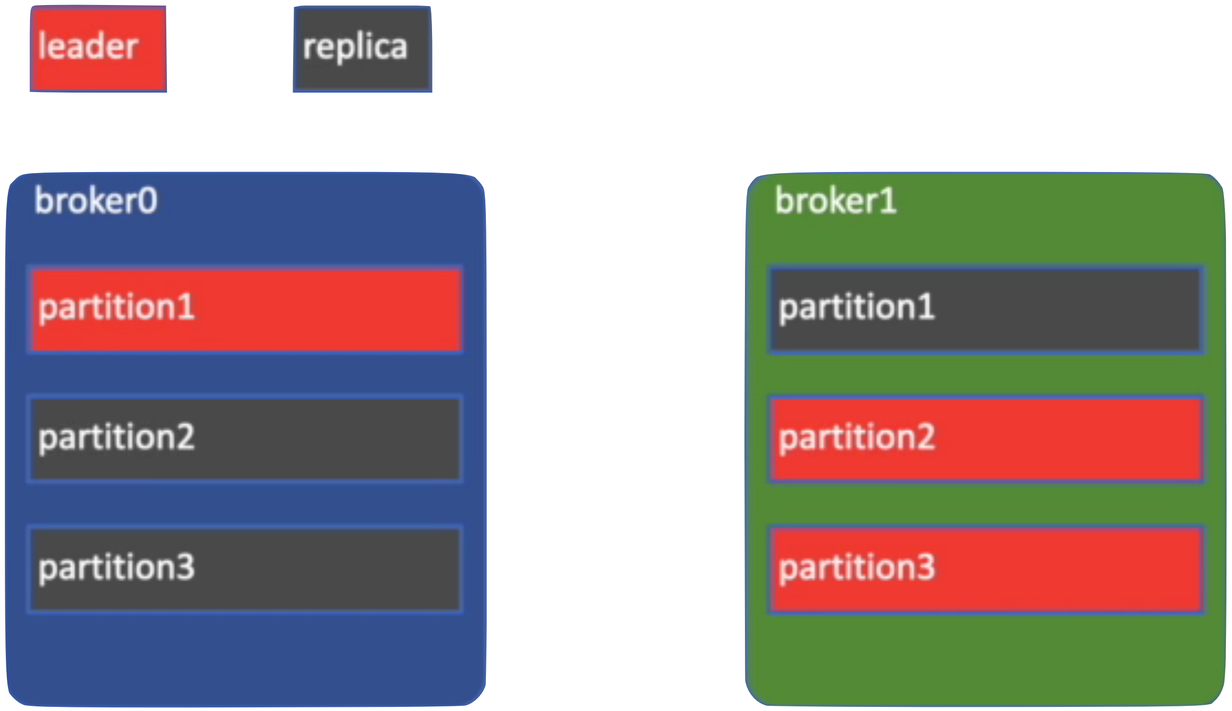
compression

producerId/epoch

baseOffset

〉 Topic/partition/offset - адрес записи в кластере

Структура кластера



Broker – каждый из серверов в Kafka:

〉 Обслуживает topic partitions

〉 Возможна репликация (leader + followers)

〉 Можно иметь несколько копий партиций в каждом топике

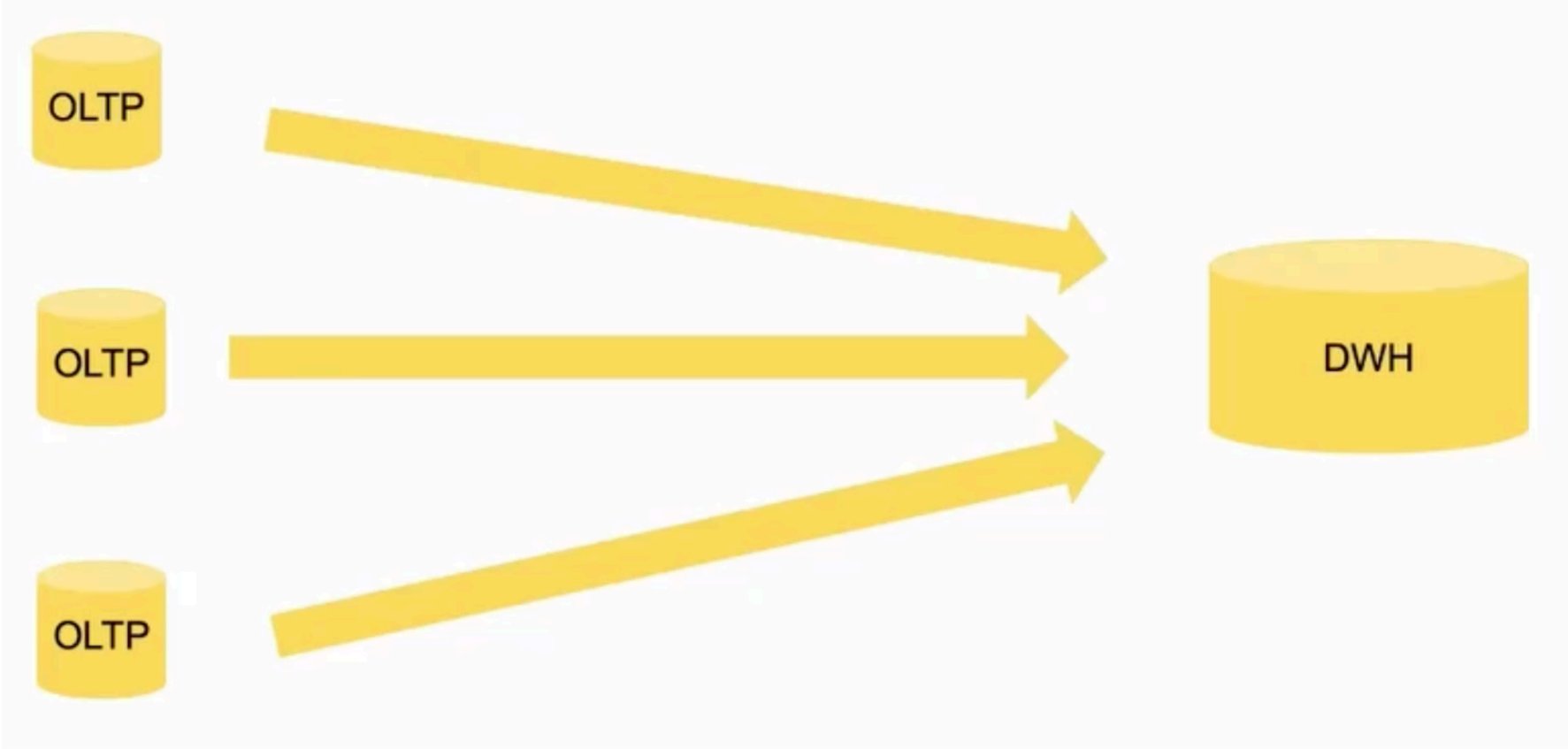
Демо 1

2 - Захват изменений с источника

Мотивация

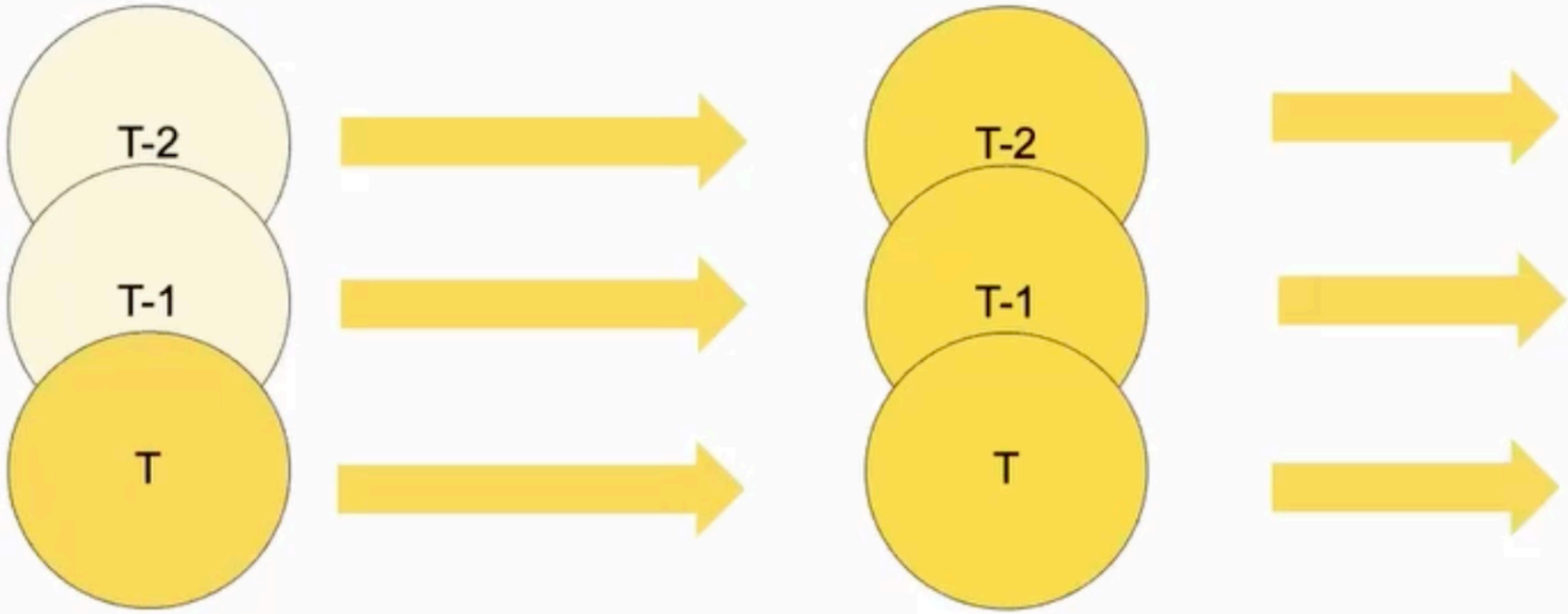
〉 У нас есть OLTP-база или ODS-слой в хранилище
〉 Нужно забрать из этих слоёв данные и как-то положить их в хранилище
〉 Лучше, если мы будем это делать не полным пересчетом таблицы
〉 Такой процесс называется **захват изменений с источника**, или **Change Data Capture (CDC)**

Change Data Capture



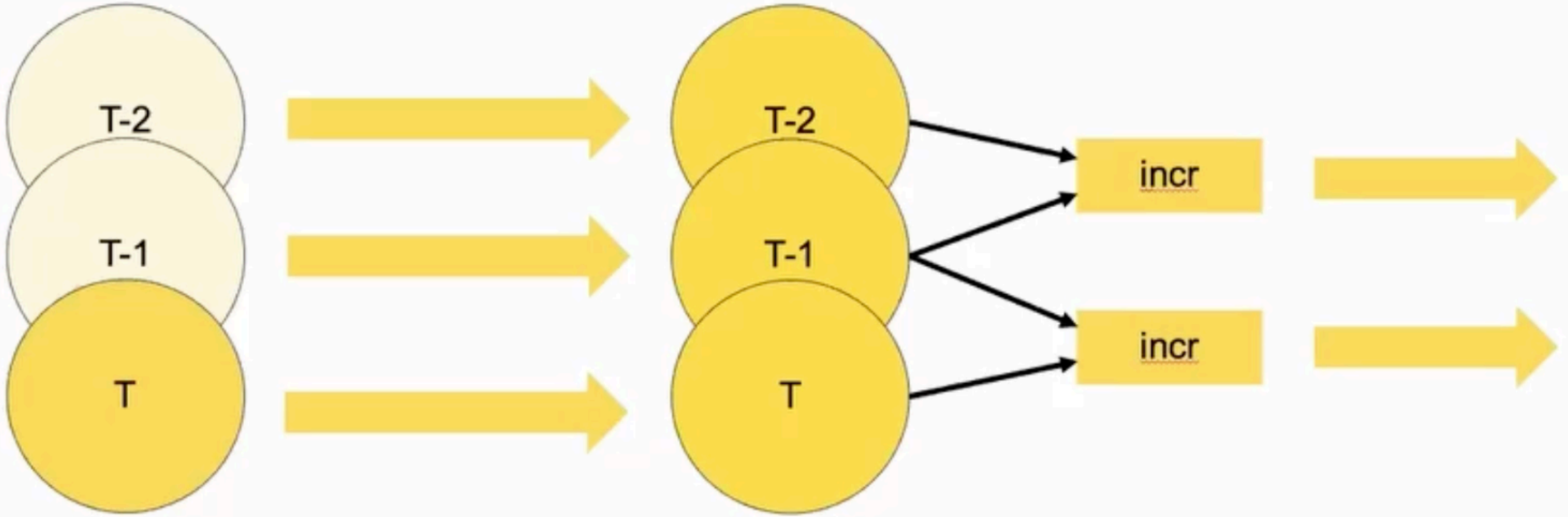
Snapshot

Каждый запуск ETL процесса мы захватываем все данные из таблицы-источника



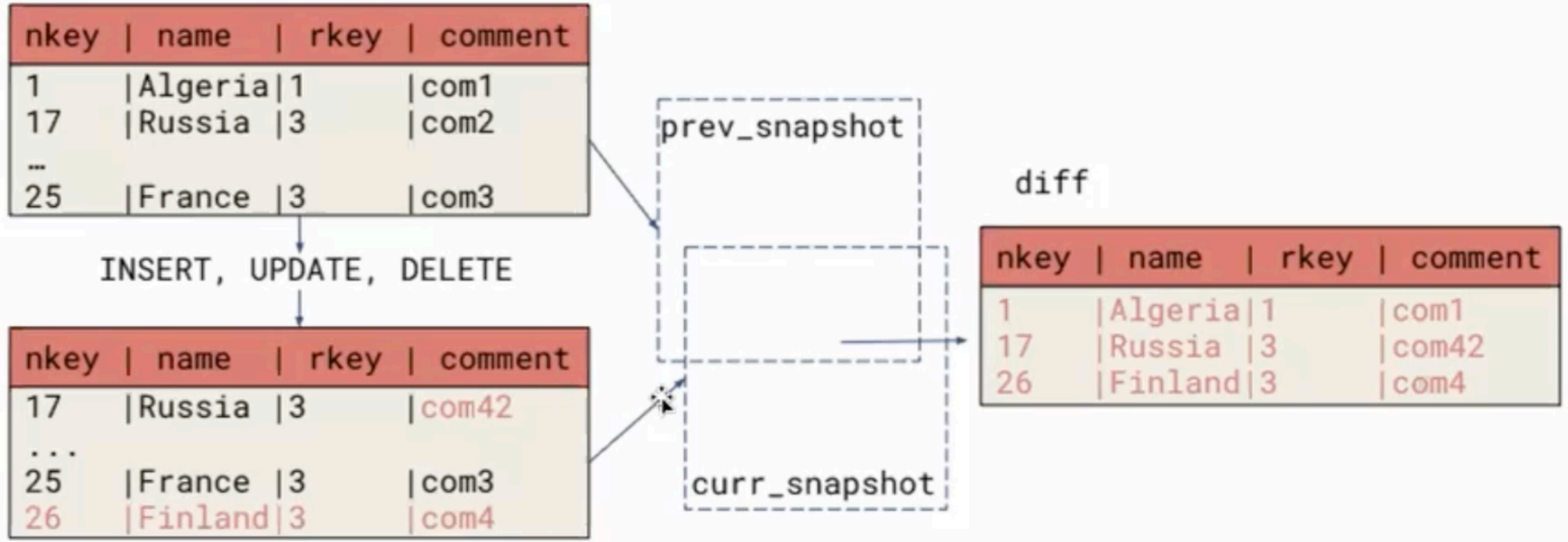
Diff от snapshot

Каждый запуск ETL процесса мы захватываем все данные из таблицы-источника



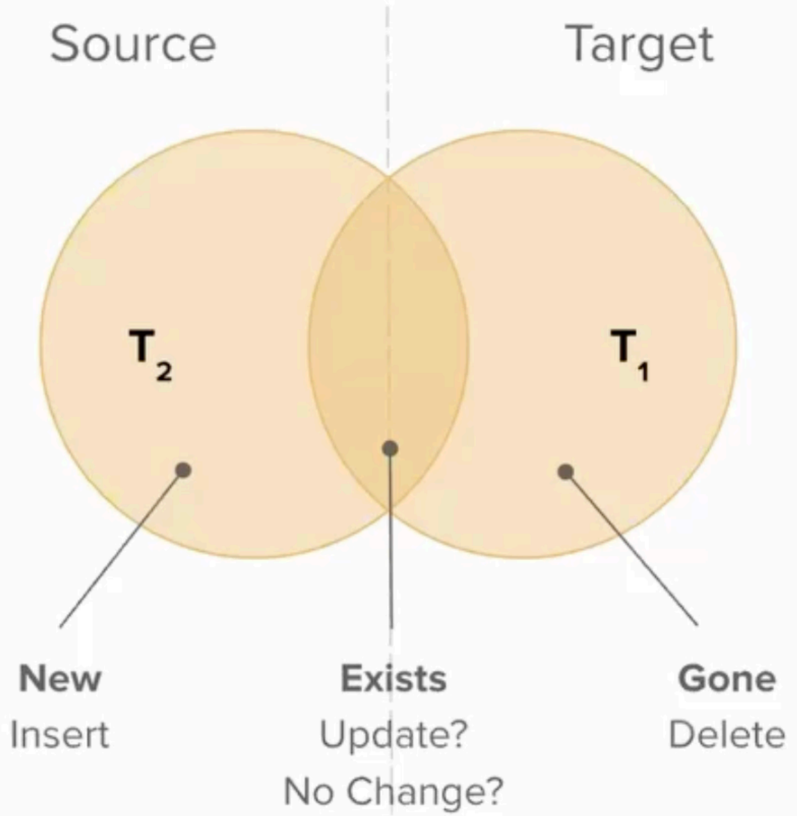
Diff от snapshot

Каждый запуск ETL процесса мы захватываем все данные из таблицы-источника



Diff от snapshot

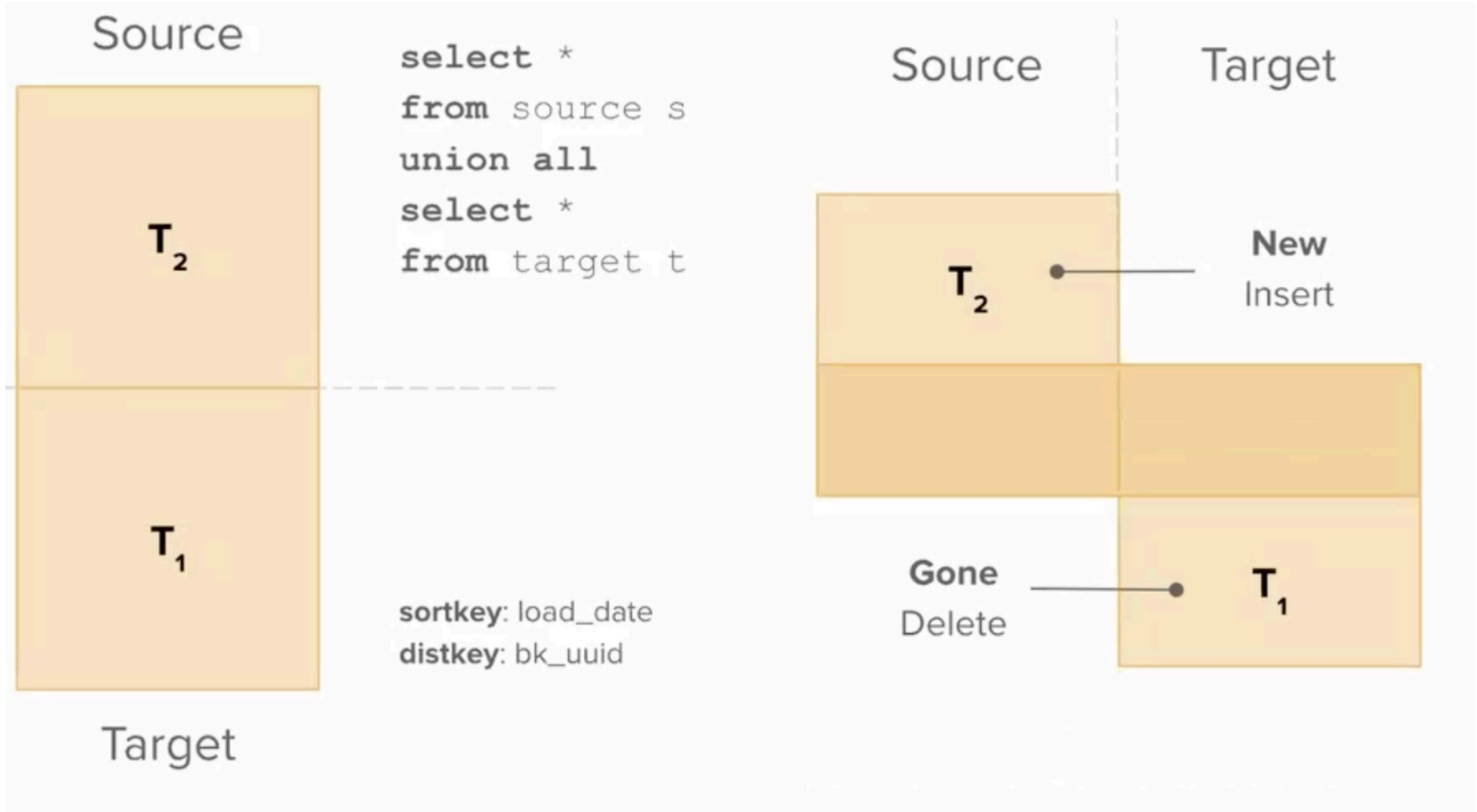
Поиск diff через join




```
SELECT *
FROM SOURCE S
FULL JOIN TARGET T
ON S.BK = T.BK
```

```
CASE
WHEN T.BK IS NULL THEN 'insert'
WHEN S.BK IS NULL THEN 'delete'
WHEN T.BK = S.BK AND S.HASH <> T.HASH
THEN 'update'
ELSE 'no-change'
END
```

Diff от snapshot

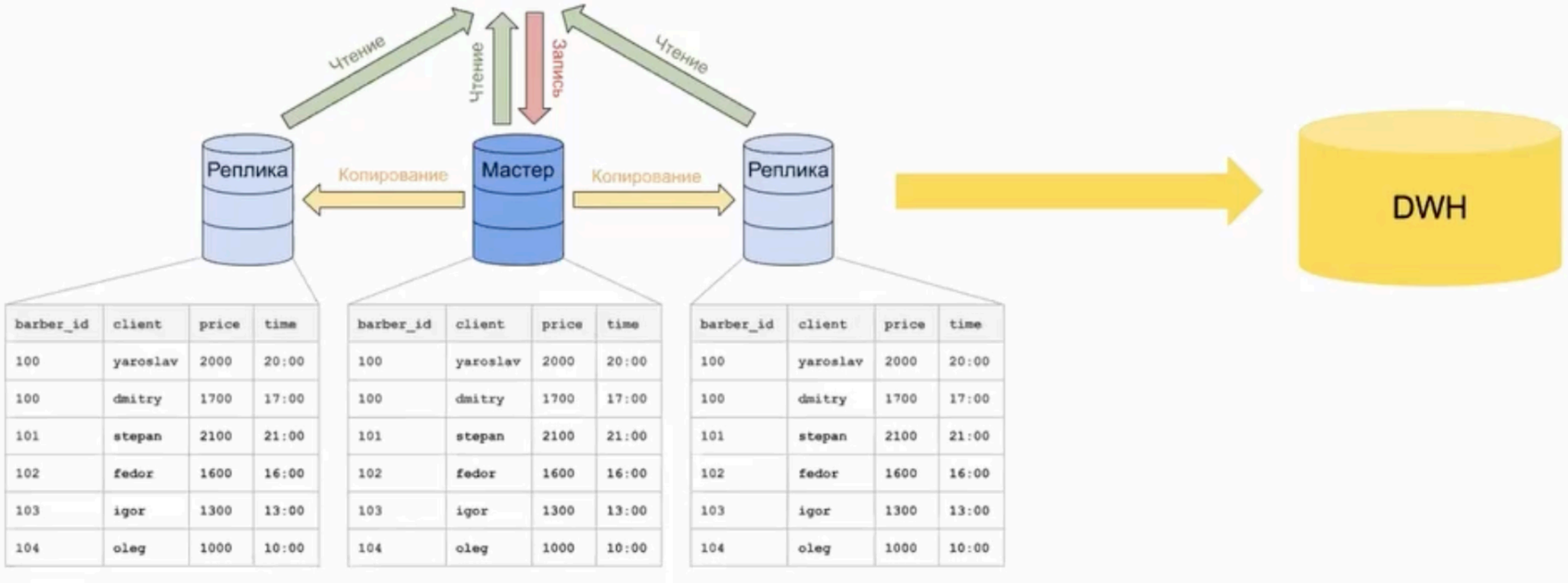


Особенности diff-метода

〉 Требуется дополнительное место для хранения snapshot
〉 Требуются большие вычислительные ресурсы для больших наборов данных
〉 Делается относительно просто
〉 Обрабатывает delete

Вспоминаем, о чем уже говорили

Для уменьшения нагрузки на основную OLTP базу используется репликация



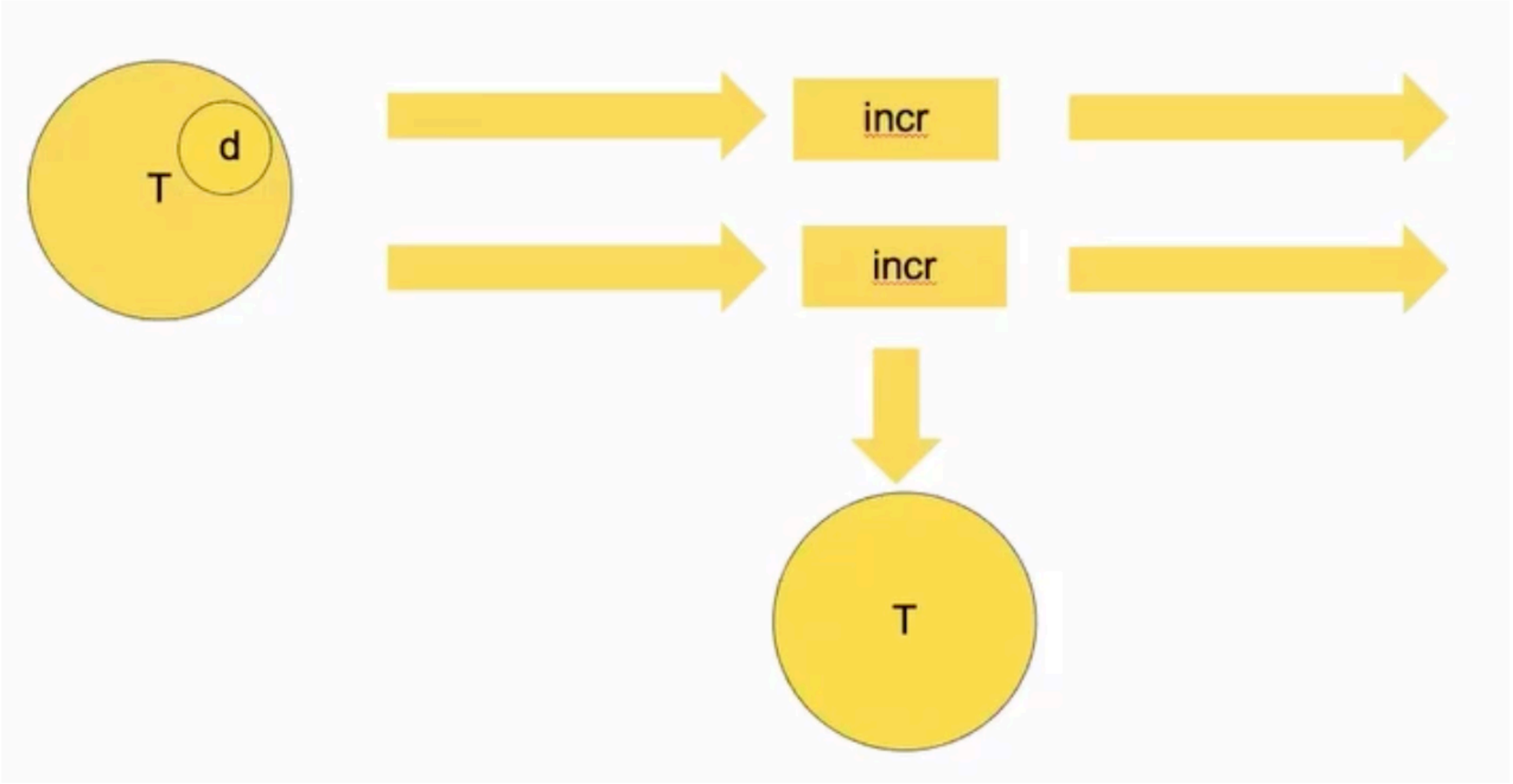
barber_id	client	price	time
100	yaroslav	2000	20:00
100	dmitry	1700	17:00
101	stepan	2100	21:00
102	fedor	1600	16:00
103	igor	1300	13:00
104	oleg	1000	10:00

barber_id	client	price	time
100	yaroslav	2000	20:00
100	dmitry	1700	17:00
101	stepan	2100	21:00
102	fedor	1600	16:00
103	igor	1300	13:00
104	oleg	1000	10:00

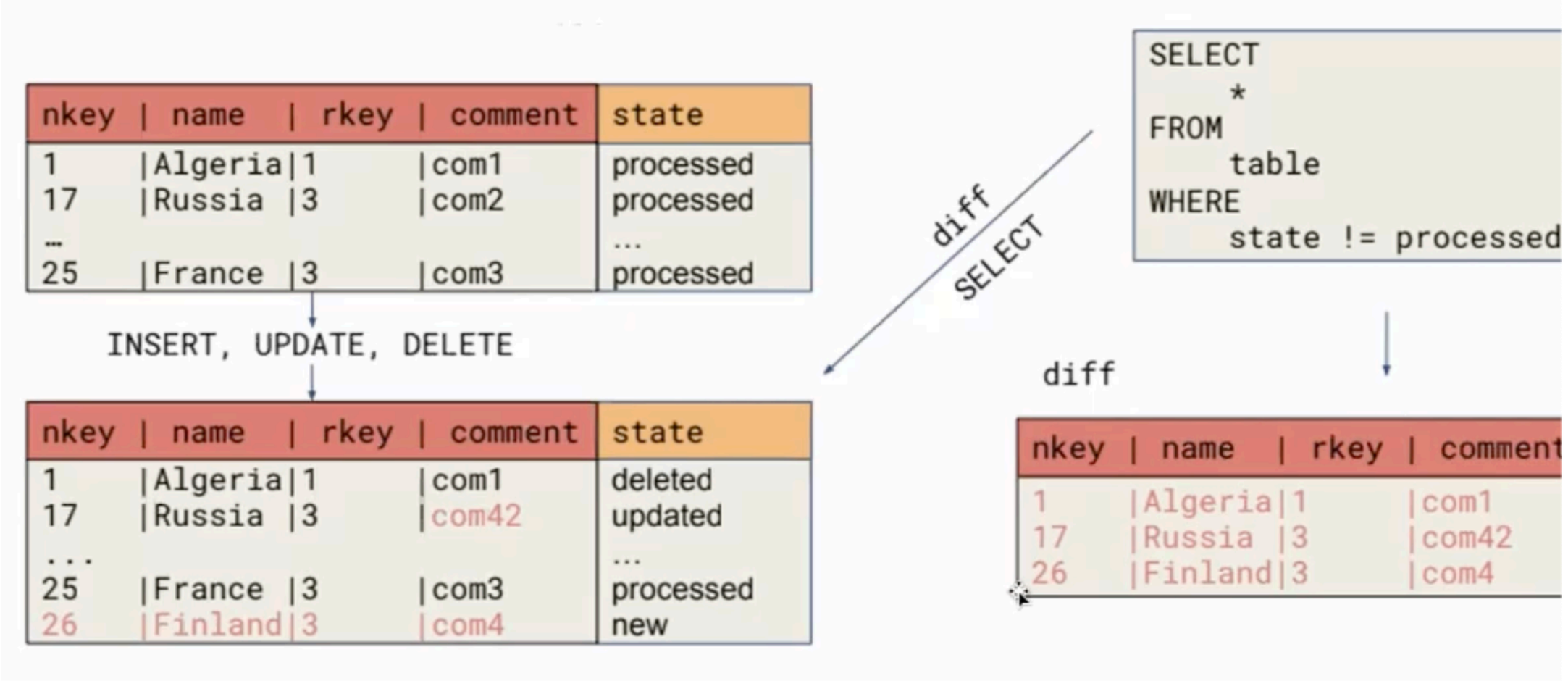
barber_id	client	price	time
100	yaroslav	2000	20:00
100	dmitry	1700	17:00
101	stepan	2100	21:00
102	fedor	1600	16:00
103	igor	1300	13:00
104	oleg	1000	10:00

Версионирование

Каждый запуск ETL-процесса мы захватываем только те данные из таблицы-источника, которые изменили свое состояние



Версионирование



Версионирование

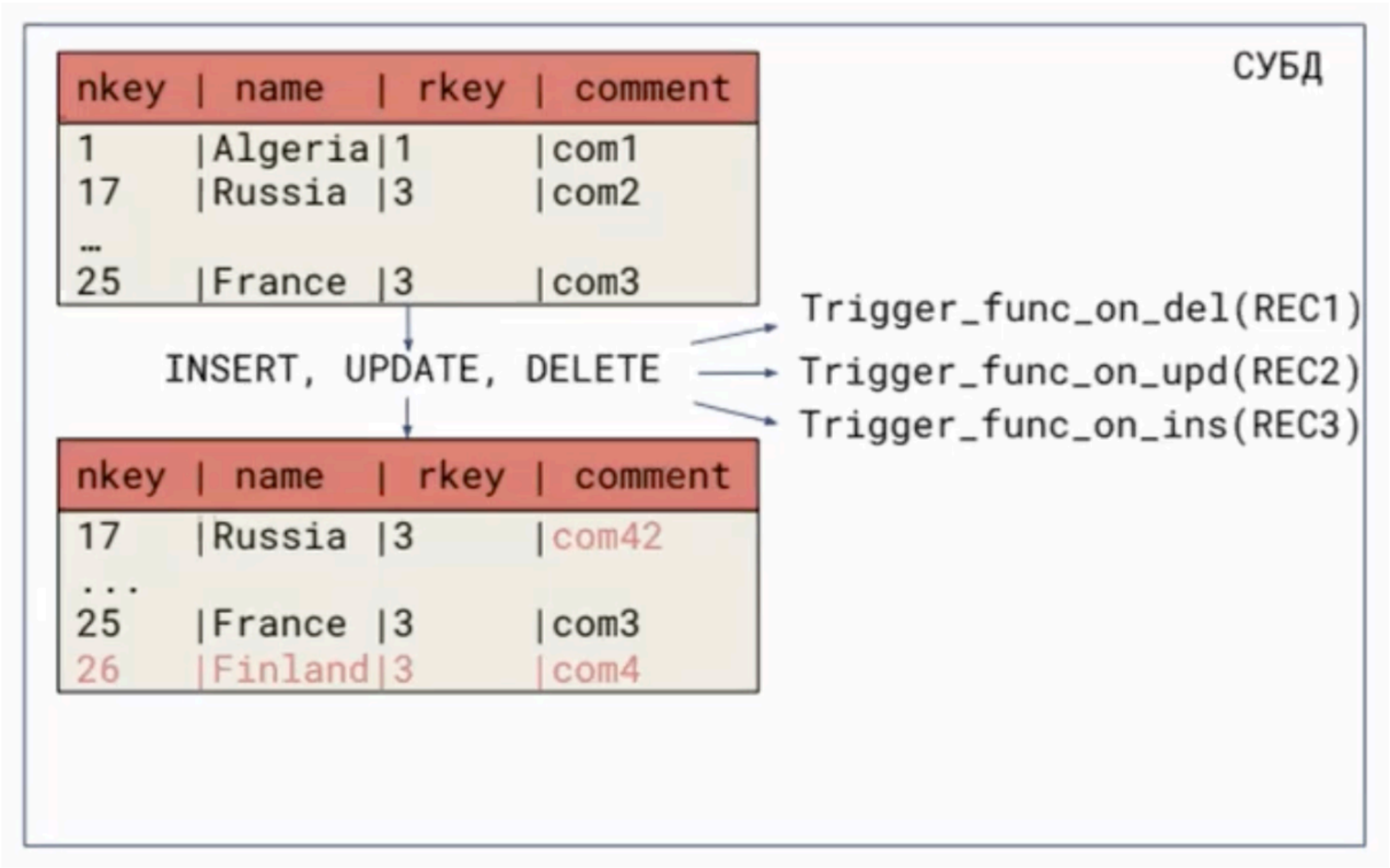
〉 Дополнительные колонки во всех схемах таблицы
〉 Отложенный delete - если не реализован soft delete, то придется брать факт delete из сверки snapshot’ов
〉 Не работает с truncate

Версионирование

〉Дополнительные колонки во всех схемах таблицы
〉Отложенный delete - если не реализован soft delete, то придется брать факт delete из сверки snapshot’ов
〉Не работает с truncate

Дополнительные риски:
〉Потеря строк по watermark: долгая транзакция коммитит строку с updated_at раньше уже захваченного max, и строка пропускается
〉Потеря промежуточных значений

Триггеры



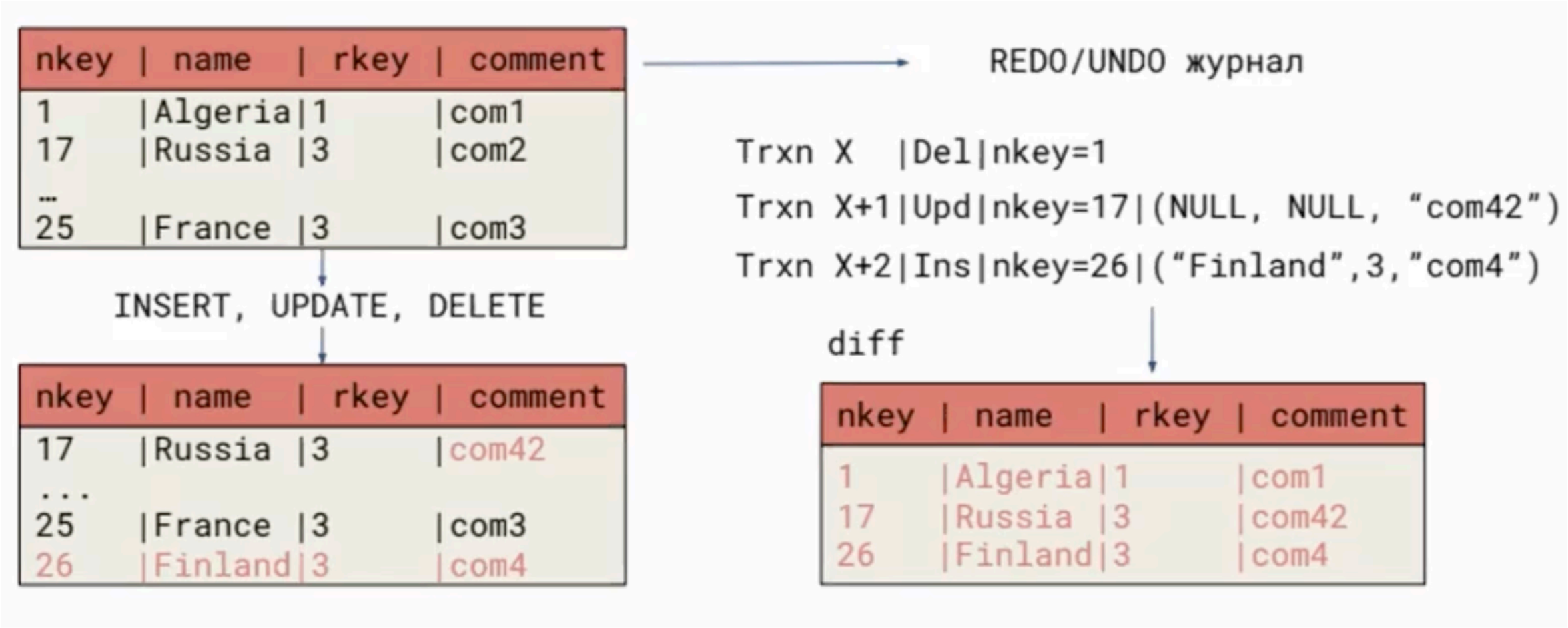
〉 Можно пушить во внешнюю очередь (если это позволяет СУБД)

〉 Можно пушить в отдельную табличку

Триггеры

〉 Поддерживается ограниченный набор действий
〉 Триггер выполняется **синхронно внутри бизнес-транзакции**. Это добавляет latency, плюс ошибка в триггере роняет транзакцию
〉 Дополнительные дисковые операции
〉 Не работает / ограниченно работает с truncate
〉 Сложно настраивать и поддерживать

Журналы транзакций



Журналы транзакций

〉 Сложно использовать из-за разных форматов журналов
〉 Асинхронный метод
〉 Отражает последовательность транзакций
〉 Нет дополнительной нагрузки на СУБД (но есть риск заполнения диска при неправильном max_wal_size)

Что делать?

Самое лучшее, что можно сделать - настроить log-based CDC (WAL / binlog / redo)

Если так по какой-то причине сделать нельзя:

〉Для всех объектов завести триггерный механизм, или специальные поля

〉Создать реплику для снижения нагрузки на OLTP

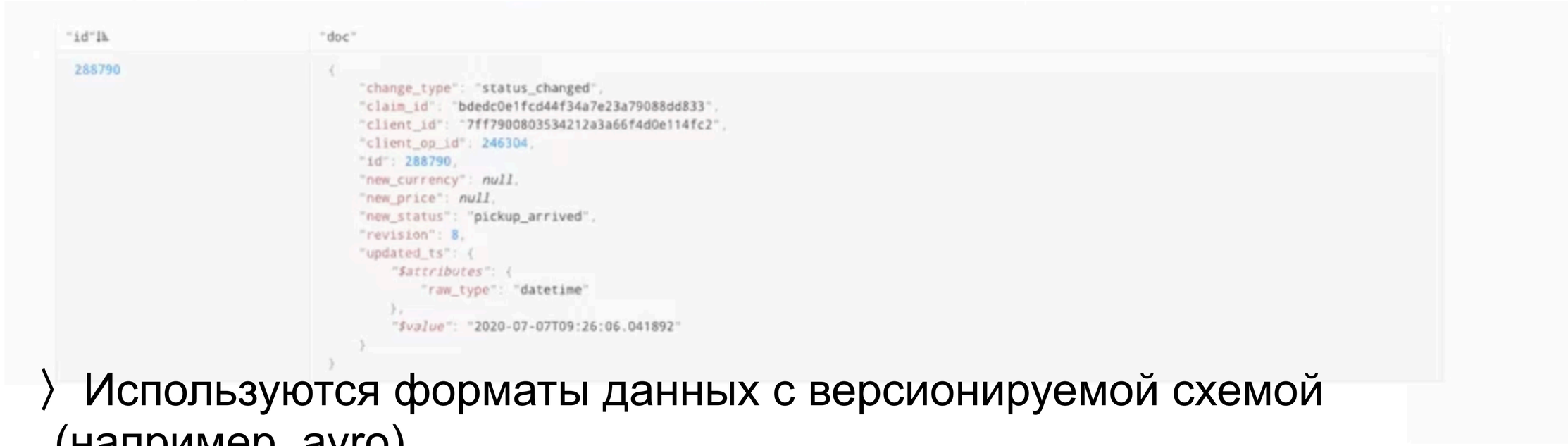
〉Повесить индексы

Проблема дрифта схемы

Состав полей в таблице тоже может меняться

〉 Изменения по логам в некоторых СУБД позволяет отслеживать удаление полей и появление новых полей

〉 В RAW слое все данные сериализуются из табличного вида в структуру ключ-значение



〉 Используются форматы данных с версионируемой схемой (например, avro)

3 - Debezium

Вспоминаем блок про CDC

Особенности CDC на журналах транзакций:

Плюсы:

- Асинхронный метод
- Минимальная дополнительная нагрузка на СУБД
- Отражает последовательность транзакций
- Все изменения, включая delete

Минусы:

- **Сложно использовать из-за разных форматов журналов**

Решение проблемы

Давайте придумаем тулзу, которая умеет читать журналы транзакций популярных СУБД, и будет их преобразовывать в один формат ¯_(ツ)_/¯

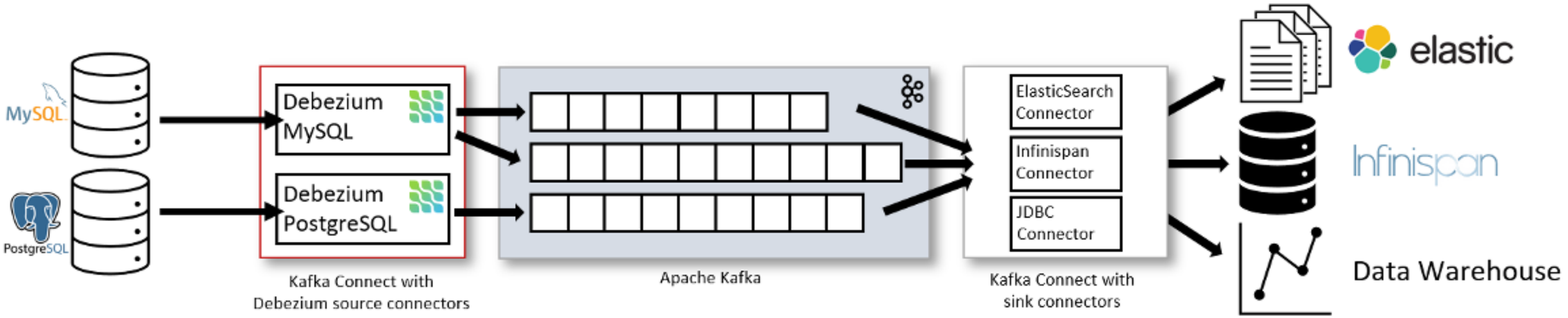
Решение проблемы

Давайте придумаем тулзу, которая умеет читать журналы транзакций популярных СУБД, и будет их преобразовывать в один формат ¯_(ツ)_/¯

И такую тулзу придумали
Она называется **Debezium**



Как работает Debezium



Демо 2